EJERCICIOS 1 - WIDGETS

Jorge Fernández

2º GS DAM Trinitarias - Salamanca

Índice

Ejercicio 1 El detective de árboles 2
Ejercicio 2 ¿Con estado o sin estado? 5
Ejercicio 3. El contador que no cuenta 8
Ejercicio 4. Los datos bajan, los eventos suben 10
Ejercicio 5. Revisor de código de la IA 13

Ejercicio 1 El detective de árboles

Análisis propio

En mi dibujo usé amarillo para la estructura y rojo para lo visible. Puse dos Row para cifras y etiquetas, olvidé el botón y traté child y radius como si fueran widgets.



Figura 1. Mi análisis de la pantalla.

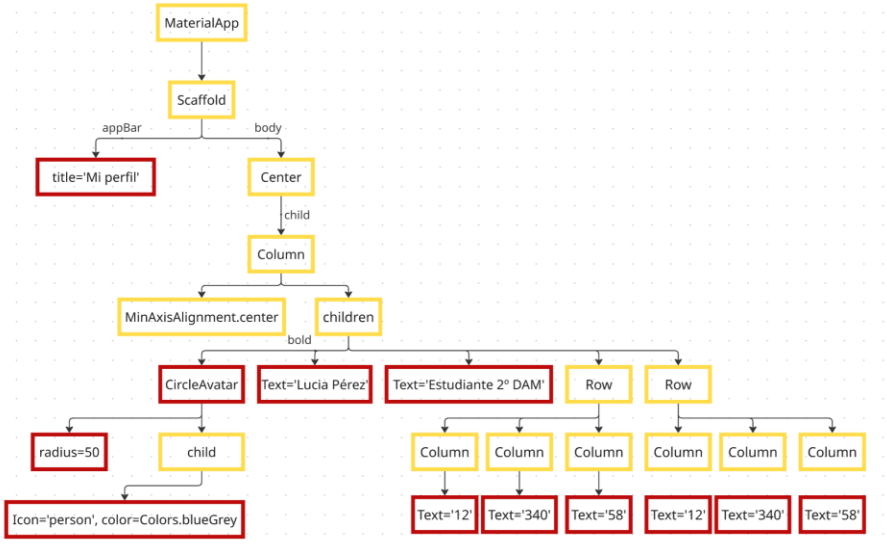


Figura 2. Mi árbol inicial, antes de la IA.

La parte repetida son las tres parejas de cifra y etiqueta. Cambian los textos, no la estructura.

Revisión de mi dibujo

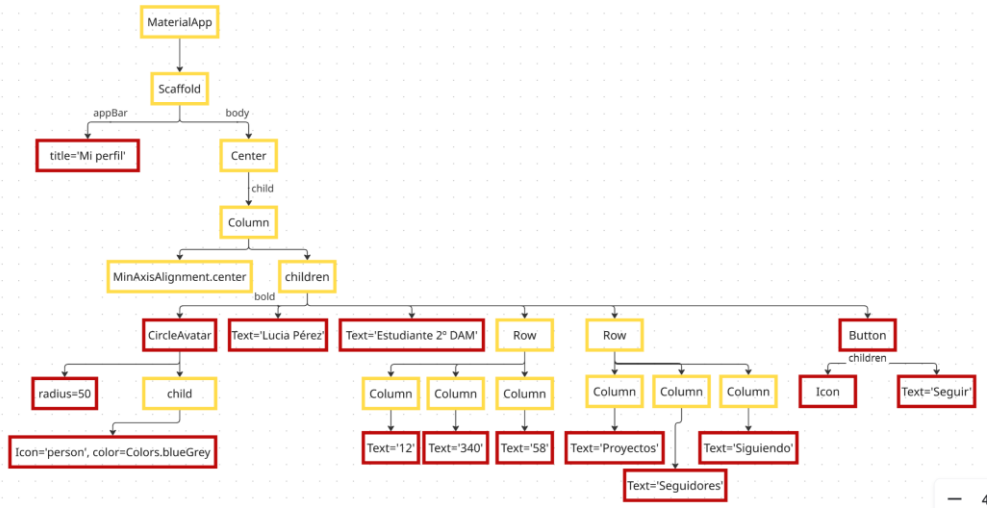


Figura 3. Añadí el botón a mi árbol y separé números y etiquetas; todavía conservé dos Row.

Diálogo con la IA

Herramienta utilizada: Codex de OpenAI.

Antes de enseñar mi esquema, escribí este prompt: «Dame tú un esquema y cuando lo hayas hecho mira si así sería más o menos en la captura que puse del word. No sé cómo poner ese div con más divs metidos dentro, cada uno centrado y una sola línea de divs».

La IA propuso un AppBar, una Column centrada y un Row con tres DatoPerfil. También añadió el botón que faltaba en mi árbol.

Árbol que propuso la IA

```
MaterialApp
└─ Scaffold
   ├─ AppBar → Text("Mi perfil")
   └─ Center → Column
      ├─ Container circular → Icon(person)
      ├─ Text("Lucía Pérez")
      ├─ Text("Estudiante de 2.º DAM")
      ├─ Row
      │  ├─ DatoPerfil("12", "Proyectos")
      │  ├─ DatoPerfil("340", "Seguidores")
      │  └─ DatoPerfil("58", "Siguiendo")
      └─ OutlinedButton → Row(Icon, Text("Seguir"))
DatoPerfil → Padding → Column(Text(valor), Text(etiqueta))
```

Comparación de los árboles

Parte	Mi propuesta	Propuesta de la IA
Estructura	MaterialApp, Scaffold, Center y Column.	Coincide y concreta AppBar con Text.
Estadísticas	Dos Row: cifras por un lado y etiquetas por otro.	Un Row con tres DatoPerfil; cada uno agrupa cifra y etiqueta.
Avatar	CircleAvatar con Icon.	Container circular con Icon.
Botón	Faltaba en mi primer árbol; luego dibujé Button.	OutlinedButton con Icon y Text.

Prefiero el Row único: cada cifra queda junto a su etiqueta. Mantengo CircleAvatar porque me da un círculo de color para el icono sin configurar la forma de un Container; es un widget fuera de la lista aprobada pero como justifico su uso, es usable.

Para pasar al código pedí: «ahora quiero que escribas el código y el ej1.dart, modifica el código de main cada vez que cambiemos de ejercicio para que el .dart sea correlativo a su ejercicio. Una vez tengamos ya todo, escribes el código con lo que hemos acordado». Después ajusté por mi cuenta el tamaño del nombre y los colores del botón y de las cifras.

Al programar cambié mis dos Row por uno con tres DatoPerfil. Así puedo cambiar el aspecto de las estadísticas en un solo sitio.

Código del widget repetido

DatoPerfil recibe valor y etiqueta por el constructor.

```
const DatoPerfil({super.key, required this.valor, required this.etiqueta});
final String valor;
final String etiqueta;
```

Resultado y comprobación

La pantalla usa tres veces DatoPerfil, un StatelessWidget con valor y etiqueta finales recibidos por constructor.

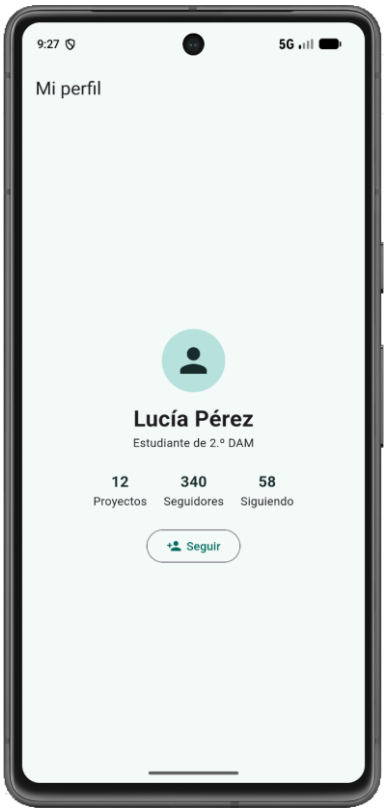


Figura 4. Pantalla funcionando.

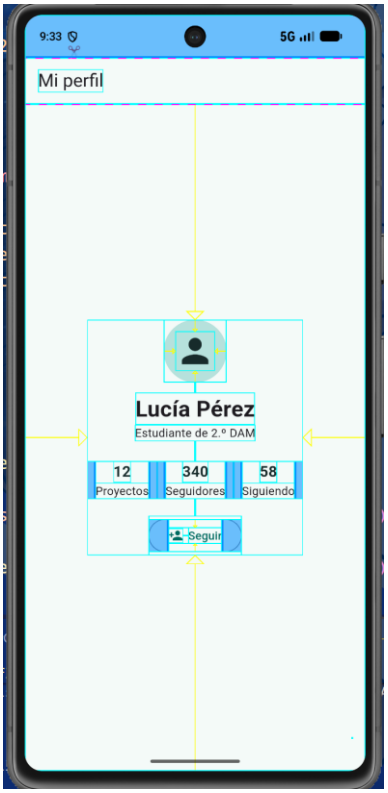


Figura 5. La misma pantalla con debugPaintSizeEnabled.

Activé debugPaintSizeEnabled con flutter run --dart-define=DEBUG_PAINT=true. Sus recuadros muestran el Row con los tres datos y el botón, como en el árbol final.

Conclusión

En este ejercicio he aprendido a distinguir widgets de propiedades y a agrupar lo que se repite, como una función de una clase para ahorrar código.

Ejercicio 2 ¿Con estado o sin estado?

Mi clasificación inicial

N.º	Descripción del widget	¿Stateless o Stateful?	¿Qué cambia? (o por qué nada cambia)
1	Un texto con el nombre de la aplicación en la barra superior.	Stateless	No cambia nada porque es un texto que no cambia.
2	Un interruptor para activar el modo oscuro de una pantalla.	Stateful	Cambia el estado interno del interruptor, de 1 a 0 o viceversa, y con ese cambio, resuelve nuevos colores para la app.
3	Una tarjeta que muestra el nombre de un contacto que recibe de su padre.	Stateless	Recibe información del padre, pero como tal no modifica información suya interna.
4	Un contador de vasos de agua que suma uno al pulsar un botón.	Stateful	Al pulsar el botón se añade uno al texto, así se está cambiando todo el rato y teniendo nuevos valores cada vez que se pulsa el botón.
5	Una estrella de «favorito» que se rellena al tocarla.	Stateful	Al pulsar la Estrella, cambian sus valores
6	Un botón que recibe por parámetro la función que debe ejecutar al pulsarlo.	Stateful	Recibe una función, y dependiendo de ella adopta una forma u otra
7	Un reloj que muestra la hora y se actualiza cada segundo.	Stateful	Cambia el valor del reloj cada segundo
8	Un campo de texto que muestra debajo lo que vas escribiendo.	Stateless	Pasa de estar vacío a pasarle los parametros que se reciben en otro campo; recibe el estado de que ha escrito tal y modifica el texto escribiendo tal

Diálogo con la IA

Para contrastar mi tabla utilicé ChatGPT. Le escribí este prompt sin enseñarle mis respuestas: «tengo que hacer el ejercicio dos, dame una tabla hecha, yo ya tengo la mía, pero quiero que hagas tú la tuya sin tener en consideración la mía.»

Clasificación de ChatGPT (las descripciones completas están en mi tabla):

Caso	ChatGPT	Motivo que dio
1	StatelessWidget	El título es fijo y no guarda estado propio.
2	StatefulWidget	Guarda si el modo oscuro está activado y actualiza la pantalla.
3	StatelessWidget	Recibe el nombre del padre y solo lo muestra.
4	StatefulWidget	Guarda los vasos y aumenta el contador al pulsar.
5	StatefulWidget	Recuerda si la estrella está marcada.
6	StatelessWidget	Ejecuta la función recibida sin guardar estado.
7	StatefulWidget	Actualiza periódicamente la hora que muestra.
8	StatefulWidget	Conserva el texto escrito para reflejarlo debajo.

Comparación y desacuerdos

Las respuestas que me dio son mucho más cortas pero directas al grano. Coincidimos en seis casos. Pero discrepamos en el botón con callback y en el campo de texto que muestra lo escrito.

Caso	Mi respuesta	ChatGPT	Decisión razonada
6	Stateful	Stateless	El botón solo llama al VoidCallback. Si algo cambia, el estado puede estar en el padre.
8	Stateless	Stateful	El conjunto debe guardar lo escrito. Solo el Text hijo puede ser Stateless si recibe el valor del padre.

Dos ejemplos en una pantalla

Para ver quién tenía razón, en ej2.dart muestro juntos TarjetaContacto, sin estado propio, y ContadorAgua, que guarda vasos.

```
const TarjetaContacto({super.key, required this.nombre});
final String nombre;

int vasos = 0;

void sumarVaso() {
  setState(() {
    vasos++;
  });
}
```

Figura 6. Tarjeta y contador en la misma pantalla; tras tres pulsaciones, el contador muestra 3.



Esquema del cambio de estado

ContadorAgua guarda un número de vasos. Al pulsar el botón, sumarVaso llama a setState() y el texto muestra el nuevo valor.

```
vasos = 0
   ↓ pulsar «Añadir vaso»
sumarVaso() → setState(() { vasos++; })
   ↓
build() → Text('Vasos de agua: $vasos')
```

Preguntas y conclusión

Un StatelessWidget puede mostrar datos nuevos si el padre se los pasa. Por ejemplo, TarjetaContacto puede recibir otro nombre sin guardar estado propio.

Hacer todo StatefulWidget ocultaría quién es responsable de cada cambio. Aquí el contador recuerda los vasos; la tarjeta solo muestra un nombre.

Aprendí que mostrar un valor no es lo mismo que guardar su estado: el botón ejecuta un callback y el ejemplo del campo de texto necesita conservar lo escrito en algún widget.

Ejercicio 3. El contador que no cuenta

Predicción y comprobación del código original

Antes de ejecutar predije que, tras tres pulsaciones, ambos textos seguirían en 0. Creí, equivocadamente, que inicializar las variables a 0 impedía cambiarlas porque se quedaría algún cambio con ello y no dejaría modificarlo.

Al ejecutarlo en el móvil vi 0/0 al inicio y también tras tres pulsaciones. Acerté el resultado, pero no la causa: un int iniciado en 0 sí puede cambiar.

Ciclo de actualización

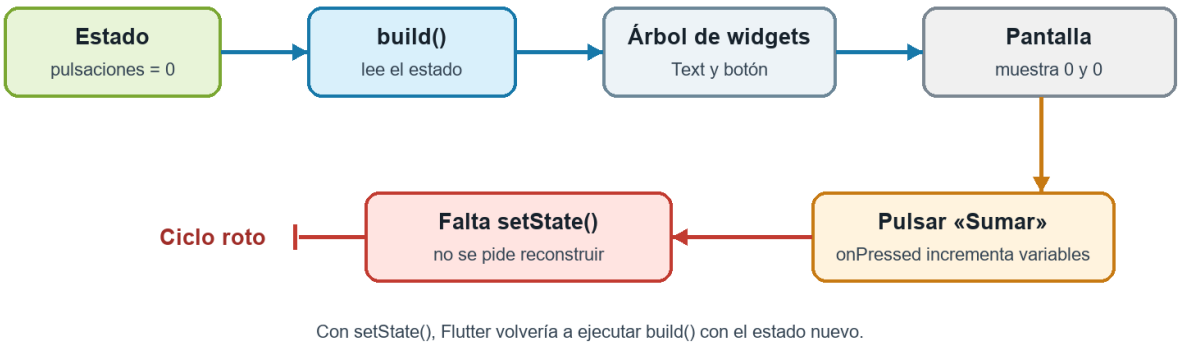


Figura 7. Ciclo de la figura 5 del enunciado, completado para este contador.

El primer hueco corresponde a los widgets que devuelve build(). El ciclo se rompe tras onPressed: las variables cambian, pero sin setState() Flutter no reconstruye la pantalla.

Consulta a la IA y valoración

Herramienta: Codex, de OpenAI. Prompt enviado: «ahora, sin leer el word, explicame qué está pasando, no me des el código corregido ».

Codex encontró dos fallos: falta setState() en onPressed y total se declara dentro de build(), por lo que volvería a 0 en otra reconstrucción. pulsaciones sí está en State.

La IA corrigió mi explicación y vio el segundo fallo. El problema visible al pulsar es la ausencia de setState(); el reinicio de total aparecería en una reconstrucción posterior.

Corrección y resultado

Con la explicación de Codex, dejé ambas variables en _ContadorState y puse los incrementos dentro de setState().

Línea original	Línea nueva	Motivo
int total = 0; dentro de build()	int total = 0; como atributo de _ContadorState	Así el valor se conserva entre reconstrucciones y no vuelve a 0 cada vez que se llama a build().

pulsaciones++;
total++;
	setState(() {
	pulsaciones++;
	total++;
	});
		setState() avisa a Flutter del cambio y provoca que los Text se reconstruyan con los nuevos valores.

```
int pulsaciones = 0;
int total = 0;

onPressed: () {
  setState(() {
    pulsaciones++;
    total++;
  });
},
```

Volví a ejecutar la aplicación con esta corrección del código y pulsé «Sumar» tres veces. Los dos valores llegaron a 3, como muestra la captura del emulador.



Figura 8. Contador corregido después de tres pulsaciones: 3 y 3.

Preguntas y conclusión

Flutter no reconstruye la pantalla por cambiar una variable cualquiera. En este caso, setState() avisa del cambio y build() vuelve a leer los valores.

Una variable local de build() se crea de nuevo en cada llamada. Un atributo de State conserva su valor mientras existe ese estado; por eso moví total allí.

Gracias a este ejercicio he aprendido que cambiar el dato y actualizar lo que se ve son dos pasos distintos.

Ejercicio 4. Los datos bajan, los eventos suben

Mi diseño inicial

Antes de pedir código, dibujé un padre con tres hijos. El padre guardaba una bandera 0/1 y un contador inicial de 12; al tocar el corazón, sumaría o restaría uno.

Parte	Lo que puse en el primer esquema
Padre	Guarda la bandera del corazón y el contador. Decide si suma o resta.
Corazón	Recibe la bandera para mostrar el icono marcado o vacío.
Texto	Pensé que recibiría la indicación de aumentar o disminuir.
Reiniciar	Pensé que devolvería los contadores a su valor inicial.

Acerté al poner el estado en el padre, pero pensé que el texto recibiría una orden de sumar o restar. En realidad recibe el número ya calculado, al principio será 12, y al reiniciarlo, sería 0 y el corazón desmarcado.

Prompt enviado a la IA

Herramienta: Codex, de OpenAI. Ya conocía mi esquema por la conversación anterior. En el mensaje para generar la aplicación le pasé la imagen de referencia y este prompt:

«Tendrás que crear una app en flutter, te paso las Restricciones y lo que debe hacer la aplicación:
Qué debe hacer la pantalla
• Al pulsar el corazón, se marca (relleno) o se desmarca (vacío), y el número de «me gusta» sube o baja en uno.
• El texto «A N personas les gusta esto» muestra siempre el número correcto.
• El botón «Reiniciar» pone el contador a cero y deja el corazón desmarcado.
Restricciones
• En toda la pantalla solo puede haber un StatefulWidget.
• El botón del corazón, el texto del contador y el botón «Reiniciar» deben ser tus propios StatelessWidget.
• Ningún widget hijo puede llamar a setState() ni modificar el estado directamente.
haz una aplicación que sea similar a la imágen que te he pasado»

Tomé como estado inicial el de la imagen: corazón marcado y 12 «me gusta». Para la zona de imagen utilicé Container e Icon, que están entre los widgets permitidos.

Revisión del código propuesto

Comprobé las seis restricciones. Las líneas citadas son de ej4.dart; la corrección posterior no cambió su estructura.

Comprobación	¿Sí?	Línea que lo demuestra
Un único StatefulWidget	Sí	Publicacion: 9; _PublicacionState: 16.
Tres hijos StatelessWidget propios	Sí	BotonMeGusta: 68; TextoMeGusta: 95; BotonReiniciar: 109.
Atributos de los hijos final	Sí	marcado y alPulsar: 75-76; cantidad: 98; alPulsar: 112.
Funciones recibidas por constructor	Sí	alPulsar en 69-73 y 110; llamadas en 83 y 116.
setState() solo en el padre	Sí	Métodos cambiarMeGusta y reiniciar: 20-32.
Solo widgets permitidos	Sí	MaterialApp, Scaffold, AppBar, Padding, Column, Container, Center, Icon, Row, IconButton, Text, SizedBox y OutlinedButton: líneas 3-116.

Esquema final y funcionamiento

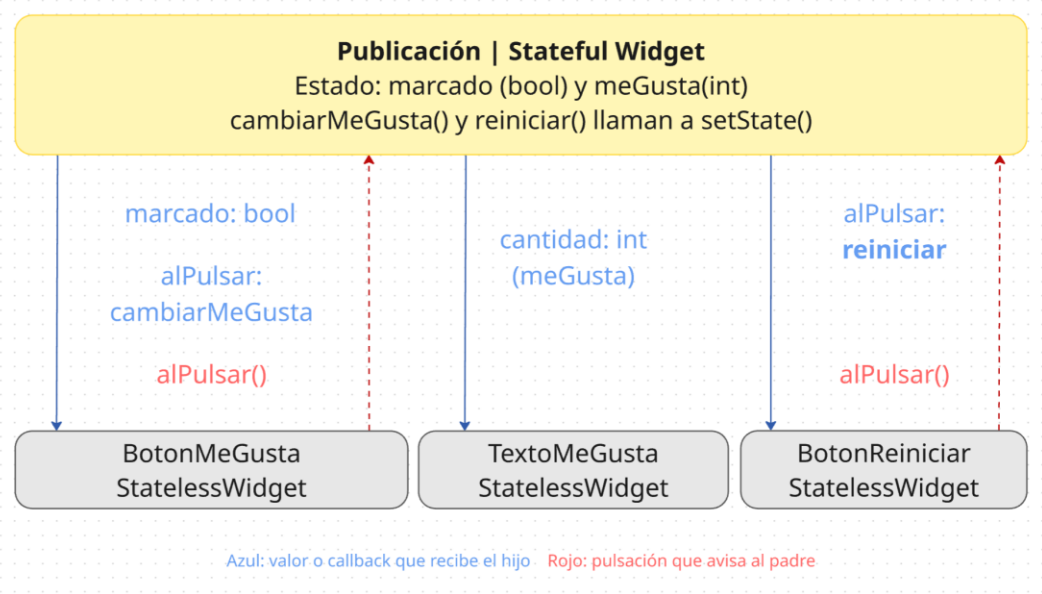


Figura 9. Estado en Publicacion y comunicación con sus tres hijos.

Publicacion guarda marcado y meGusta. El corazón recibe el booleano, el texto recibe la cifra y ambos botones avisan al padre mediante funciones.

En mi primer esquema no dibujé esos avisos y traté el texto como si recibiera una orden. El esquema final distingue datos que bajan y eventos que suben.

Código principal y comprobación

La parte que cambia el estado está concentrada en _PublicacionState:

```
bool marcado = true;  int meGusta = 12;
void cambiarMeGusta() { setState(() { marcado = !marcado; meGusta += marcado ? 1 : -1; }); }
void reiniciar() { setState(() { marcado = false; meGusta = 0; }); }
```

En el emulador comprobé 12/marcado → 11/vacío → 12/marcado → 0/vacío al reiniciar.

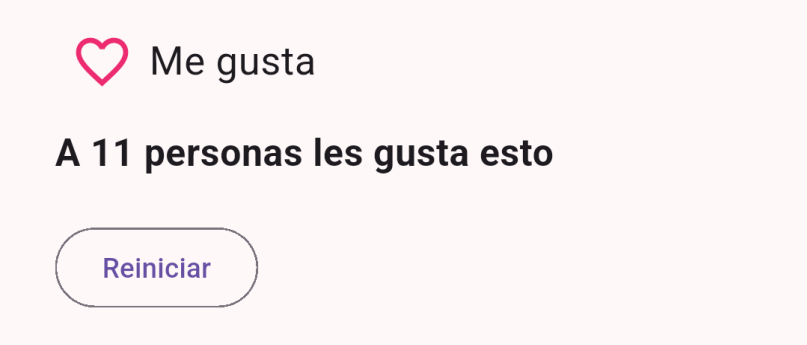


Figura 10. Tras tocar el corazón: 11 «me gusta» y corazón vacío.

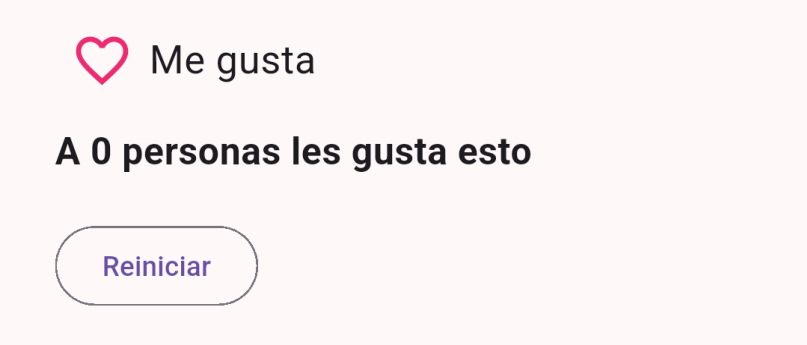


Figura 11. Tras pulsar Reiniciar: 0 «me gusta» y corazón vacío.

Preguntas y conclusión

Si corazón y texto guardaran estados separados, podrían mostrar valores distintos. Al vivir ambos en Publicacion, cambian juntos.

Los hijos solo notifican la pulsación y por eso pueden reutilizarse. Publicacion decide si suma, resta o reinicia.

Ahora entiendo mejor el flujo: los datos bajan del padre y los avisos de pulsación suben. La prueba confirmó que el corazón y el contador siguen sincronizados.

Ejercicio 5. Revisor de código de la IA

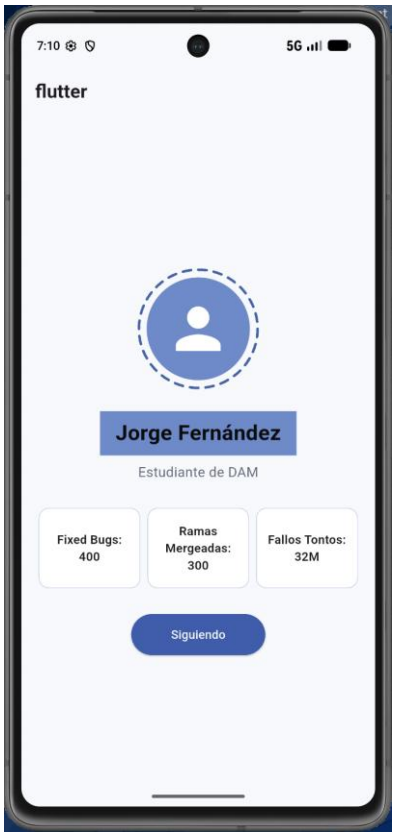
Mi petición y la primera versión

Este fue mi prompt para la primera versión de ej5.dart. Fijé el contenido, los colores y el botón como único widget con estado:

“Ahora, para el ejercicio 5, tendrás que crearme otra aplicación de flutter. Tendremos el appBar que pondrá alineado a la izquierda (como siempre) un texto en la negrita original que pone flutter. En el body todo estará alineado en el centro. Como primer objeto tendremos un icono grande con el borde del icono en vez de ser uniforme será por rayas, y dentro el círculo será de color #6f8ac9. Un objeto más abajo estará el nombre y el curso. El nombre (Jorge Fernández) estará destacado (fondo de un color #6f8ac9 y letra #0c0c0c; y el otro texto de abajo no estará destacado y tendrá una letra gris que no la tape el fondo más pequeña que ponga "Estudiante de DAM". Otro objeto más abajo tendremos una fila con 3 datos (cada dato separado del otro, como tarjetas), en los cuales, en el primero pondrás "Fixed Bugs: 400", en el segundo pondrá "Ramas Mergeadas: 300" y en la tercera pondrá "Fallos Tontos: 32M", y con eso ya terminamos las cartas de "presentación". Y como último objeto pondremos un botón de seguir, fondo #6f8ac9 y la letra blanca que ponga seguir. Después miraré yo el código y veré si hay algo mal, el qué está mal y cómo lo puedo solucionar. Cabe recalcar que solo puedes utilizar un único stateful widget que será el botón de seguir, ya que este es el único que cambiará al hacerle click y el fondo cambiará a un azul un poco más oscuro y el texto cambiará a "Siguiendo".”

Concreté textos, cifras y colores para poder revisar la primera versión punto por punto. Pedí un solo StatefulWidget porque el cambio de «Seguir» a «Siguiendo» era la única interacción. El borde rayado fue una idea visual que luego descarté al ver el resultado y revisar su código.

Captura de la primera versión



Mi revisión y los cambios que haría

Requisito	Qué observo	Qué modificaría
Título «flutter» a la izquierda y en negrita	Pedí «flutter», pero queda genérico.	Lo cambiaría por «Mi Aplicación».
Icono grande, borde rayado y círculo #6f8ac9	Respeta el color y el borde que pedí.	Quitaría las rayas: no sabría explicar ese código. Dejaría la prueba comentada.
Nombre Jorge Fernández destacado y curso debajo	Nombre y curso se ven como pedí.	Pondría en blanco la letra del nombre; el curso está bien.
Tres tarjetas separadas con los datos pedidos	Usa Expanded; no lo conocía.	Preguntaría para qué sirve y si hay otra opción.
Botón «Seguir» y cambio a «Siguiendo»	Cambia texto y color al pulsarlo.	Lo mantendría, pero preguntaría cómo funciona.
Un único StatefulWidget	Solo el botón tiene estado.	Lo mantendría.
Código que puedo explicar y justificar	Entiendo casi todo, salvo el borde, Expanded y el cambio de color.	Preguntaría por esas tres partes antes de aceptarlas.

Explicaciones de Codex

Expanded reparte el ancho disponible de Row entre las tres tarjetas. Lo mantuve porque se adaptan juntas al ancho de pantalla. SingleChildScrollView permite desplazarse si falta altura.

El borde rayado usaba CustomPaint y Canvas para dibujar arcos. No me gustó el resultado ni sabría explicar bien esa parte, así que pedí dejar la prueba comentada. El icono circular sigue visible.

El botón guarda el booleano siguiendo. Al pulsarlo, setState() invierte su valor; dos expresiones ? : eligen el texto y el color. Ambos cambios ya estaban en mi prompt.

CircleAvatar ya lo habíamos usado en clase para fotos de perfil. Aquí muestra el icono dentro de un círculo de color.

Dos estados del botón

Probé el botón en el emulador: pasa de «Seguir» a «Siguiendo» y oscurece el azul.



Figura 12. Antes de pulsar: «Seguir».



Figura 13. Después de pulsar: «Siguiendo».

Auditoría de la primera versión

Después de mi revisión, contrasté la primera versión con cada requisito. Las líneas indican dónde quedó esa misma estructura en el código final.

Requisito	¿Cumple?	Evidencia y valoración
AppBar con título	Sí	Primera captura; AppBar en ej5.dart, líneas 28–33. Después cambié el texto.
Icono en círculo de color	Sí	Primera captura; avatar e icono en las líneas 57–65.
Nombre y curso	Sí	Primera captura; textos en las líneas 67–85. Luego cambié la letra a blanco.
Tres datos en Row	Sí	Primera captura; Row y tres Expanded en las líneas 88–96.
Seguir / Siguiendo	Sí	Dos capturas del emulador; bool y setState() en las líneas 165–183.
Un StatefulWidget	Sí	BotonSeguir, línea 158; la parte con estado empieza en la 165.
Widgets permitidos o justificados	Parcial	Expanded, SingleChildScrollView, CircleAvatar y CustomPaint no se explicaban al principio. Los tres primeros están justificados; el borde con CustomPaint quedó comentado.

Dos rondas de mejora

Petición	Cambio realizado	Comprobación	Valoración
1. Quitar el borde rayado y revisar el título.	Quedó comentada la prueba con CustomPaint; el círculo se mantuvo. Yo cambié el título a «Mi Aplicación».	El avatar sigue visible y el análisis de Dart no encontró problemas.	Prefiero el círculo sencillo y puedo explicar el código que quedó activo.
2. Nombre en blanco; conservar Expanded y el desplazamiento.	Codex cambió el color del nombre. Las tres tarjetas siguen repartidas con Expanded y la pantalla conserva el desplazamiento.	Probé «Seguir» y «Siguiendo» en el emulador y guardé una captura de cada estado.	El nombre se lee mejor y el botón hace el cambio que pedí.

Árbol final de widgets

El árbol marca BotonSeguir como único widget propio con estado y muestra los tres TarjetaDato.

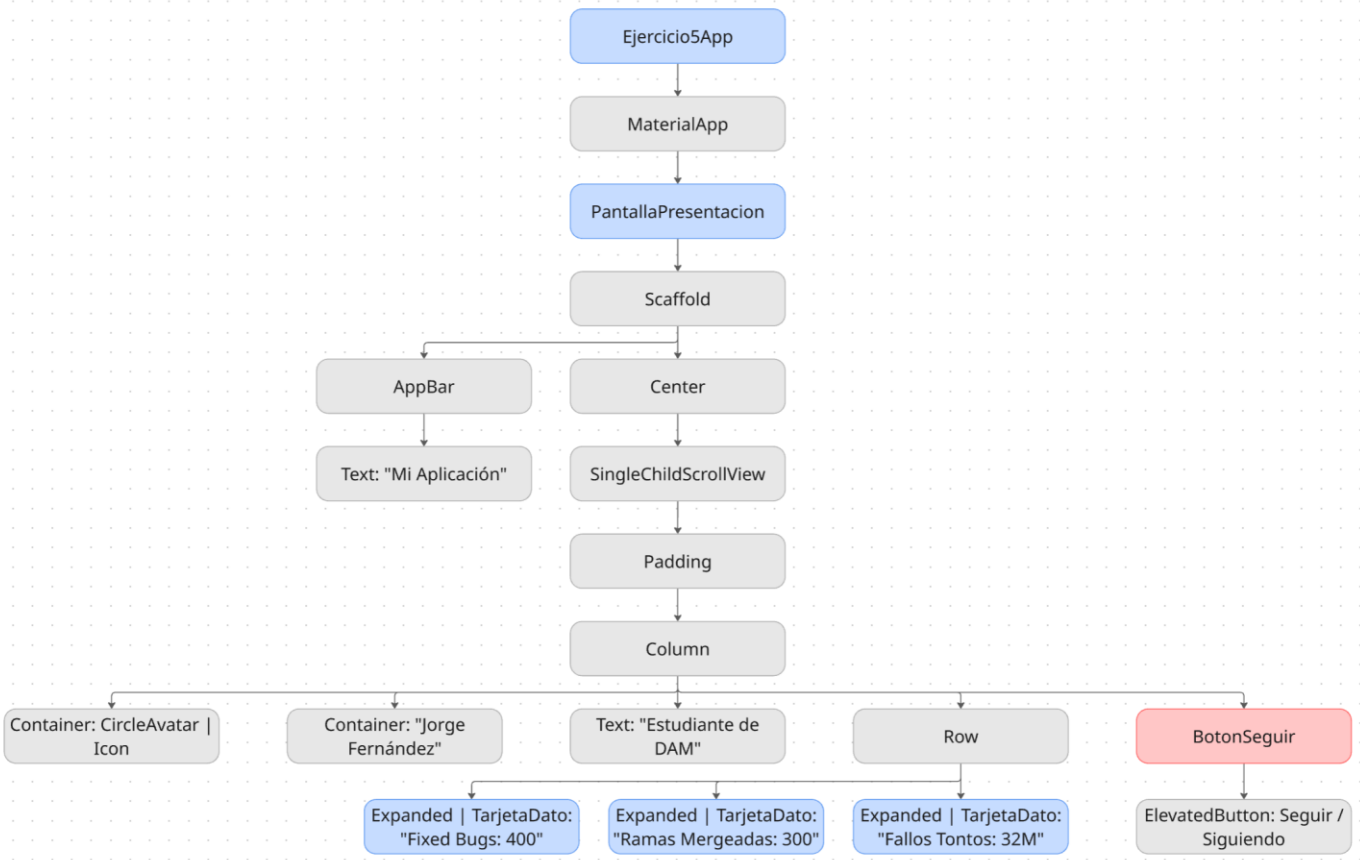


Figura 14. Árbol final del ejercicio 5, creado en Miro.

Azul: widget propio sin estado. Rojo: único widget propio con estado. Gris: widget de Flutter.

Valoración de la respuesta de la IA

La IA acertó con la composición centrada, las tres tarjetas y el botón con un solo StatefulWidget. Comprobé en el emulador sus dos estados.

Corregí tres cosas: el título «flutter» era genérico, el borde rayado no quedaba bien y prefería el nombre en blanco sobre azul.

Entiendo aproximadamente el 70 % del código: la estructura, las tarjetas y el estado del botón. Me falta práctica con CustomPainter y algunos ajustes de diseño, también me falta práctica con flutter pero se irá ganando con el tiempo.

Un prompt inicial que habría sido más claro

Habría pedido desde el principio el título «Mi Aplicación», el nombre en blanco y un círculo sin borde rayado. También habría solicitado una explicación breve de Expanded y SingleChildScrollView. Aunque así he aprendido que siempre desde el principio se puede hacer mejor y focalizar mejor a la IA en lo que debe de hacer desde el primer prompt.

Conclusión

La primera versión funcionaba, pero revisar el código y probar la pantalla me ayudó a decidir qué cambiar. El único estado propio de esta pantalla está en el botón.